Basic Instructions

1/ MOV (Move)

- Syntax: MOV destination, source
- Example: MOV R1, #10; Explanation: This instruction moves the immediate value 10 into register R1.
- MOV is only able to load the lower 16 bits (0-15) to a register. MOVT is used to load the upper 16 bits (16-31)
  2/ ADD (Addition)
- Syntax: ADD destination, operand1, operand2
- Example: ADD R2, R3, R4
- Explanation: This instruction adds the values in registers R3 and R4 and stores the result in register R2.
  3/ SUB (Subtraction)
- Syntax: SUB destination, operand1, operand2
- Example: SUB R2, R3, R4
- Explanation: This instruction subtracts the value in register R4 from R3 and stores the result in register R2.
  5/ LDR (Load from Memory)
- Syntax: `LDR destination, [address]`
- Example: `LDR R6, [R7]`
- Explanation: This instruction loads the **value** from the memory address stored in register R7 into register R6.
  6/ STR (Store to Memory)
- Syntax: `STR source, [address]`
- Example: `STR R8, [R9]`
- Explanation: This instruction stores the value in register R8 to the memory address stored in register R9.
  8/ LSL (Logical Shift Left)
- Syntax: `LSL destination, source, #shift_amount`
- Example: `LSL R13, R14, #2`
- This instruction shifts the value in register R14 left by 2 bits and stores the result in register R13.
  9/ AND (Bitwise AND)
- Syntax: AND destination, operand1, operand2
- Example: AND R15, R16, R17
- Explanation: This instruction performs a bitwise AND operation on the values in registers R16 and R17 and stores the result in register R15.

Bin to Hex mnemonics

```
17 → PC
00010001

0001|0001
→ 0x11
```

Key concepts in Assembly Language

- Mnemonics: Abbreviations or symbols representing machine instructions (e.g., MOV for move, ADD for addition).
- Registers: Special memory locations within the CPU used to store data temporarily during program execution.
- Instructions: Basic operations that can be executed by the processor (e.g., load data, perform arithmetic).
- Operands are the data or registers on which instructions operate.

For instance, in the instruction MOV AX, 5, "AX" is the destination register, and "5" is the immediate value being moved.

Assembler

- **Machine Language:** Computer program collection of numbers in the form of binary strings
- **Assembly Language:** Computer program collection of words called mnemonics
- **Assembler:** Computer program that translate the assembly language instruction into executable machine language instructions

Assembly language syntax

Syntax Element	Description	Example(s)
Mnemonics and Instructions	Represent the operation to be performed.	ADD, MOV, SUB **ADD**; Addition **MOV**; Move data **SUB**; Subtraction
Operands	Specify data or registers for the operation	ADD R1, R2, MOV R0, #5 **ADD R1, R2**; Add the contents of R2 to R1 **MOV R0, #5**; Move immediate value 5 into R0
Comments	Explanatory remarks (not executed).	; This is a comment
Labels	Mark specific locations in the program.	**START:** **MOV R0, #10**; Move immediate value 10 into R0
Directives	Provide instructions to the assembler.	**.data**; Start of the data section **.word 42**; Define a word with value 42
Registers	Denoted by names (e.g., R0, R1, ...).	**MOV R0, R1**; Move the value in R1 to R0
Immediate Values	Constants used directly in instructions.	**MOV R2, #15**; Move immediate value 15 to R2

Example Assembly Code

```
; Entry point
.global main

main:
    ; Enable clock for PORTF
    MOV R0, #0xE608
```

```
    MOVT R0, #0x400F
    MOV R1, #0x20
    STR R1, [R0]               ; R1 of address [R0]
```

- The code enables the clock for GPIO Port F.
- MOV R0, `#0xE608` and `MOVT R0`, `#0x400F` combine to form the address 0x400FE608 (> Oh, this is little-endian), which is the RCGCGPIO register (Run Mode Clock Gating Control for GPIO). This register controls the clock for the GPIO ports.
- MOV R1, `#0x20` loads the value 0x20 (bit 5) into R1, enabling the clock for Port F.
- `STR R1, [R0]` stores this value in the RCGCGPIO register, turning on the clock for Port F.

-> This line declares the main label as a global symbol, meaning it's the entry point of the program.

Datasheet: TivaC-Datasheet-1 - Register 60 (RCGCGPIO)

```
; Enable digital function for PIN PF1
    MOV R0, #0x551C
    MOVT R0, #0x4002
    LDR R1, [R0]
    ORR R1, R1, #0x02              ; 0010 Change LED 3 positions
    STR R1, [R0]
```

- The code configures Pin 1 of Port F for digital use.
- `MOV R0, #0x551C` and `MOVT R0, #0x4002` combine to form the address `0x4002551C`, which is the GPIO Digital Enable (`551C`) register for Port F.
- `LDR R1, [R0]` loads the current value of this register.
- `ORR R1, R1, #0x02` enables Pin PF1 (Red-LED) for digital I/O by setting bit 1. The value `0x02` is used here to enable the pins required for the LED control.
- `STR R1, [R0]` stores the modified value back into the GPIODEN register, enabling Pin 1 for digital functionality.

Datasheet: TivaC-Datasheet-1 - Register 18 (GPIODEN)

```
; Set PIN 1 as an output
    MOV R0, #0x5400
    MOVT R0, #0x4002
    LDR R1, [R0]
    ORR R1, R1, #0x02; 0010 Change LED 3 positions
    STR R1, [R0]
```

- The code sets Pin PF1 as an output.
- `MOV R0, #0x5400` and `MOVT R0, #0x4002` combine to form the address 0x40025400, which is the GPIO Direction (GPIODIR) register for Port F.
- `LDR R1, [R0]` loads the current value of the GPIODIR register.
- `ORR R1, R1, #0x02` modifies the value by setting Pin PF1 as an output.
- `STR R1, [R0]` stores the new value back into the GPIODIR register, configuring Pin PF1 as an output pin.

Datasheet: TivaC-Datasheet-1 - Register 2: GPIO Direction (GPIODIR)

```
loop:
; Toggle LED
MOV R0, #0x5038
MOVT R0, #0x4002
LDR R1, [R0]
EOR R1, R1, #0x02     ; 0010 Change LED 3 positions
STR R1, [R0]
```

- This section toggles the LED connected to Pin PF1.
- `MOV R0, #0x5038` and `MOVT R0, #0x4002` combine to form the address 0x40025038, which is the GPIO Data (GPIODATA) register for Port F. This register controls the output value of the GPIO pins.
- `LDR R1, [R0]` loads the current value of the data register.
- `EOR R1, R1, #0x02` toggles the state of Pin PF1 by applying an exclusive OR (XOR) operation with 0x02. This effectively flips the value of Pin PF1, turning the LED on or off.
- `STR R1, [R0]` writes the toggled value back to the GPIODATA register.

Datasheet: TivaC-Datasheet-1 - Register 1: GPIO Data (GPIODATA)

```
; Delay
MOV R2, #0xFFFF
MOVT R2, #0x003E
delayloop:
SUBS R2, R2, #1      ; R2 -= 1
BNE delayloop
```

- The delay loop is introduced to provide a visible toggle rate for the LED. Without a delay, the toggling would happen too quickly to notice.
- `MOV R2, #0xFFFF` and `MOVT R2, #0x003E` load the value 0x003EFFFF into register R2, which is used as the delay counter.
- `SUBS R2, R2, #1` decrements the counter by 1 in each iteration.
- `BNE delayloop` branches back to delayloop if the value in R2 is not zero, thus creating a delay.

```
; Repeat the loop
B loop
```

After the delay, the program jumps back to the loop label to continuously toggle the LED and repeat the process indefinitely.

ASSEMBLY LANGUAGE VS HIGH-LEVEL PROGRAMMING LANGUAGES

Machine Code ← Assembler ← Assembly language ← Compiler ← High-level language code

Aspect	Assembly Language	High-Level Programming
Abstraction Level	Low-level, close to hardware	High-level, abstracted
Hardware Interaction	Direct control over hardware	Indirect, through libraries
Optimization	Used for performance-critical code	Focuses on code readability
Development Time	Time-consuming, requires expertise	Faster development cycles
Readability	Less readable due to mnemonics	More human-readable code
Main Use Cases	Hardware control, optimization	Application logic, user interfaces, communication
Maintenance	Prone to errors, harder to maintain	Easier maintenance, updates

---

ROLE OF HIGH-LEVEL PROGRAMMING

- **High-level programming languages are designed for accessibility, efficiency, and readability.**
- **Portability:**
  - Cross-platform compatibility.
  - Reduced maintenance effort.
- **Productivity and Rapid Development:**
  - Abstraction of low-level details.
  - Faster development and reduced code volume.
- **Algorithm Implementation:**
  - Efficient implementation of complex algorithms and data structures.
- Readable and maintainable code.
- **Ecosystem and Community Support:**
  - Vast libraries and frameworks.
  - Active developer communities and support.
- **Debugging and Testing:**
  - Sophisticated debugging tools.
  - Enhanced testing capabilities.
- **Integration with Assembly:**
  - High-level programming often integrates with assembly for performance optimization and hardware interfacing.

SUPPORT OF ASSEMBLY LANGUAGE FOR HIGH-LEVEL LANGUAGES

- **In building embedded systems on an ARM Cortex-M4, assembly language supports high-level languages (like C) by:**
  - Providing low-level hardware control.
  - Optimizing performance-critical code.
  - Handling interrupt service routines efficiently.
  - Creating interfaces between high-level and hardware code.
  - Ensuring portability and cross-platform compatibility.
  - Aiding debugging and profiling.
  - Customizing instructions for better performance.
  - Optimizing for energy efficiency.